



BASIC NETWORK SNIFFER

PREPARED BY:

KOKO MUSHNAGA

CodeAlpha Cybersecurity Internship

July 2026

Table of Contents

- 1. Introduction**
- 2. Project Objective**
- 3. Project Description**
- 4. Tools and Technologies Used**
- 5. System Requirements**
- 6. Implementation**
- 7. Source Code Explanation**
- 8. Results and Output**
- 9. Challenges Encountered**
- 10. Security Considerations**
- 11. Conclusion**
- 12. References**

1. Introduction

Network traffic consists of packets that travel between devices connected to a network. Packet sniffing is the process of capturing and analyzing these packets to better understand how communication occurs over a network.

The objective of this project was to develop a basic network packet sniffer using Python. The application captures packets transmitted through the network and displays useful information such as the source IP address, destination IP address, and protocol used. This project provided practical experience in understanding network communication and introduced the fundamentals of packet analysis used in cybersecurity.

2. Project Objective

The objectives of this project were:

- To understand the concept of packet sniffing.
- To develop a simple network packet sniffer using Python.
- To capture live network packets.
- To display source and destination IP addresses.
- To identify network protocols used by captured packets.
- To gain practical knowledge of network monitoring.

3. Project Description

The application was developed using Python and the Scapy library. The program continuously monitors incoming and outgoing network traffic on the local machine. Whenever a packet is detected, the application extracts basic network information and displays it in the terminal.

The packet sniffer captures only basic packet information and does not modify or interfere with network communication. It is intended solely for educational purposes and network analysis in authorized environments.

4. Tools and Technologies Used

Programming Language:

- Python 3

Development Environment:

- Visual Studio Code

Libraries:

- Scapy

Operating System:

- Microsoft Windows 10/11

Additional Software:

- Wireshark (used to observe and verify captured traffic)

5. System Requirements

Hardware Requirements

- Computer with Internet connection
- Minimum 4 GB RAM

Software Requirements

- Python 3.x
- Visual Studio Code

- Scapy Python Library
- Wireshark
- Windows Operating System

6. Implementation

The implementation process consisted of the following steps:

1. Install Python.
2. Install Visual Studio Code.
3. Install Scapy using pip.
4. Create the Python project.
5. Develop the packet capturing program.
6. Execute the application.
7. Generate network traffic by browsing websites.
8. Observe captured packets.
9. Record the results.

7. Source Code Explanation

The program imports the Scapy library to capture packets from the network interface.

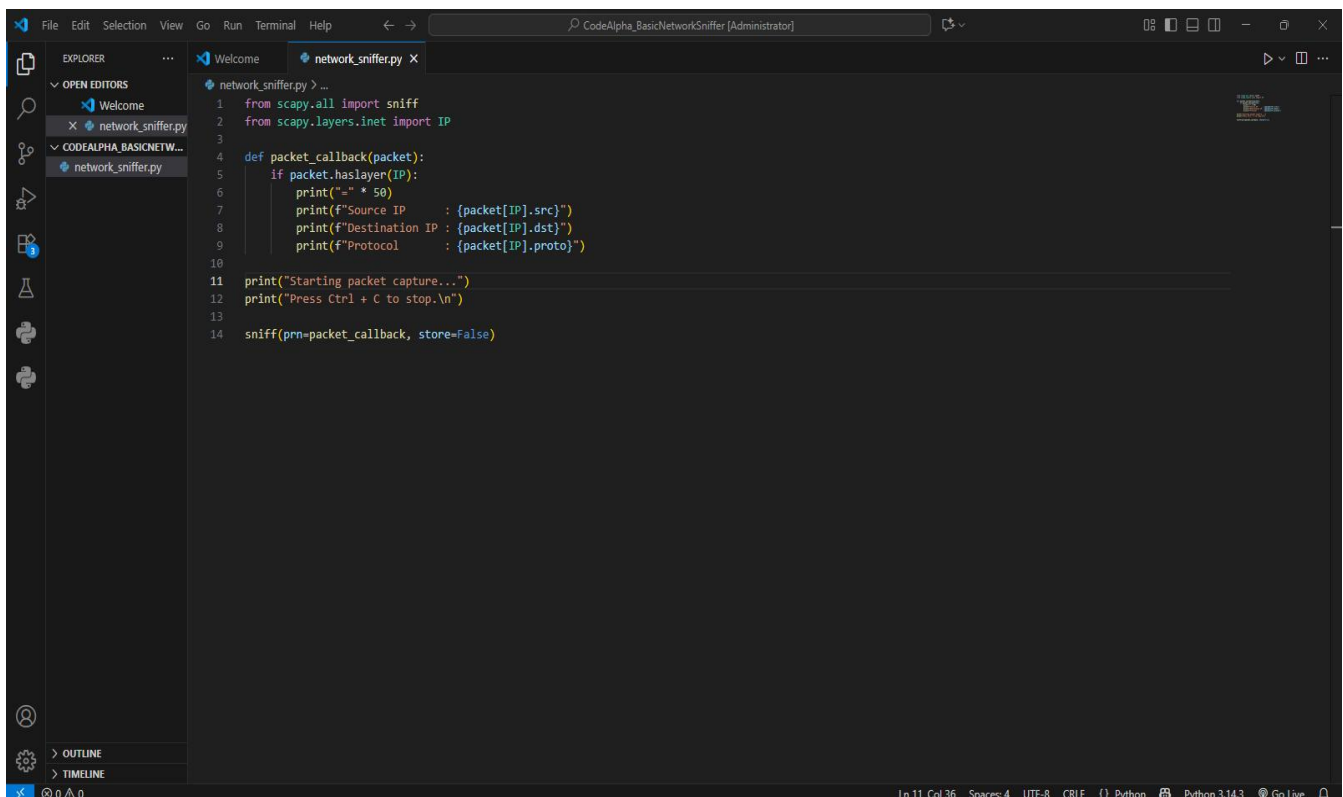
A callback function processes every captured packet.

For each packet, the program checks whether it contains an IP layer.

If an IP packet is detected, the following information is displayed:

- Source IP Address
- Destination IP Address
- Protocol Number

The sniff() function continuously captures packets until the user manually stops the program.



The screenshot shows a Visual Studio Code editor window titled 'CodeAlpha_BasicNetworkSniffer [Administrator]'. The Explorer sidebar on the left shows a project named 'CODEALPHA_BASICNETW...' with a file 'network_sniffer.py' selected. The main editor area displays the following Python code:

```
1 from scapy.all import sniff
2 from scapy.layers.inet import IP
3
4 def packet_callback(packet):
5     if packet.haslayer(IP):
6         print("-" * 50)
7         print(f"Source IP      : {packet[IP].src}")
8         print(f"Destination IP : {packet[IP].dst}")
9         print(f"Protocol      : {packet[IP].proto}")
10
11 print("Starting packet capture...")
12 print("Press Ctrl + C to stop.\n")
13
14 sniff(prn=packet_callback, store=False)
```

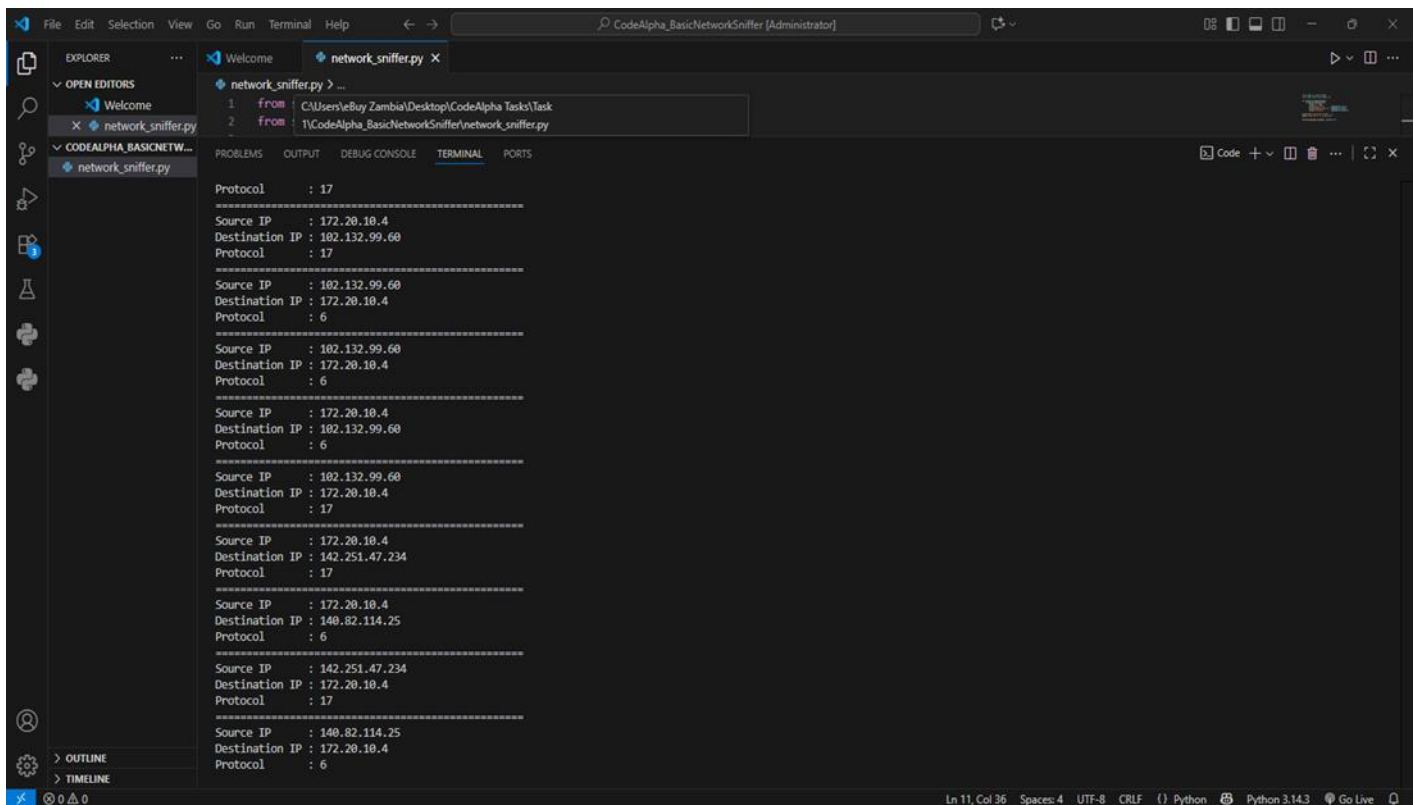
The status bar at the bottom indicates 'Ln 11, Col 36', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', 'Python 3.14.3', and 'Go Live'.

The application successfully captured live network packets generated while browsing the internet.

For each packet, the following information was displayed:

- Source IP Address
- Destination IP Address
- Protocol Number

The captured packets demonstrated how devices communicate across a network using Internet Protocol (IP).



The screenshot shows a Visual Studio Code editor with a terminal window open. The terminal displays the output of a network sniffer application, showing captured network packets. The output is formatted as follows:

```
Protocol : 17
=====
Source IP : 172.20.10.4
Destination IP : 182.132.99.60
Protocol : 17
=====
Source IP : 182.132.99.60
Destination IP : 172.20.10.4
Protocol : 6
=====
Source IP : 182.132.99.60
Destination IP : 172.20.10.4
Protocol : 6
=====
Source IP : 172.20.10.4
Destination IP : 182.132.99.60
Protocol : 6
=====
Source IP : 182.132.99.60
Destination IP : 172.20.10.4
Protocol : 17
=====
Source IP : 172.20.10.4
Destination IP : 142.251.47.234
Protocol : 17
=====
Source IP : 172.20.10.4
Destination IP : 140.82.114.25
Protocol : 6
=====
Source IP : 142.251.47.234
Destination IP : 172.20.10.4
Protocol : 17
=====
Source IP : 140.82.114.25
Destination IP : 172.20.10.4
Protocol : 6
```

The terminal window is titled "CodeAlpha_BasicNetworkSniffer [Administrator]". The editor shows the file "network_sniffer.py" open. The status bar at the bottom indicates "Ln 11, Col 36", "Spaces: 4", "UTF-8", "CRLF", and "Python 3.14.3".

9. Challenges Encountered

During implementation, several challenges were encountered:

- Installing required Python libraries.
- Understanding Scapy syntax.
- Running the application with sufficient permissions.
- Interpreting packet information.

These challenges were resolved through documentation and testing.

10. Security Considerations

Packet sniffing should only be performed on networks where permission has been granted.

Unauthorized interception of network traffic may violate privacy policies and applicable laws.

This project was conducted strictly for educational purposes using an authorized environment.

11. Conclusion

This project provided practical experience in network traffic analysis and packet capturing using Python. Through the implementation of a basic packet sniffer, it became possible to understand how data travels across a network and how cybersecurity professionals analyze communication between devices.

The project also strengthened programming skills, networking knowledge, and familiarity with cybersecurity tools used in packet analysis.